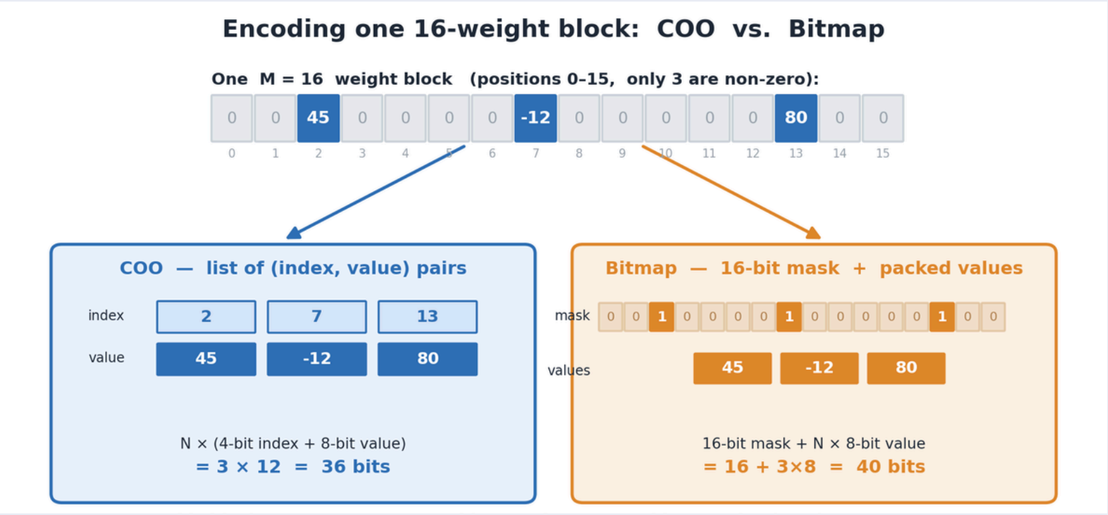
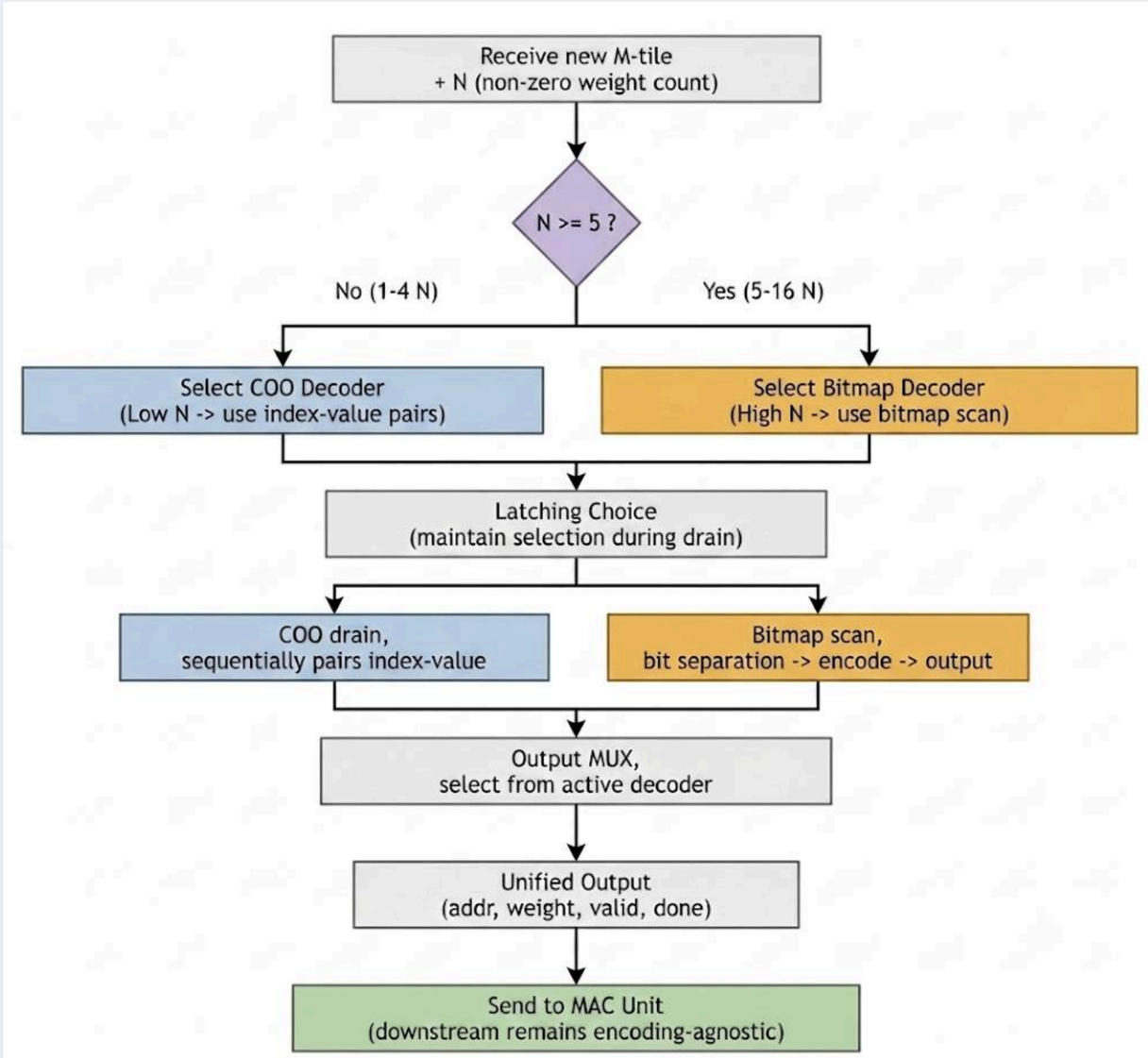
3. Implementation – Accelerator Design

Weight Decoder: Native N:M Sparse Decoding Hardware

- Dual-path decoder, auto-selected per layer by a single bit: use_bitmap = (N ≥ 5) — the same crossover derived on the theory slide.
- **COO path** (N≤4): a barrel shifter + counter sequentially extracts (position, value) pairs — 12-bit right-shift per cycle, N cycles total.
- **Bitmap path** (N≥5): a priority encoder (find-first-set) walks the 16-bit mask, clearing the lowest set bit each cycle via bmap & (bmap-1) — N cycles total.
- Both paths converge on one unified output interface (addr, weight, valid, done) — the downstream MAC array never needs to know which sub-decoder is active.

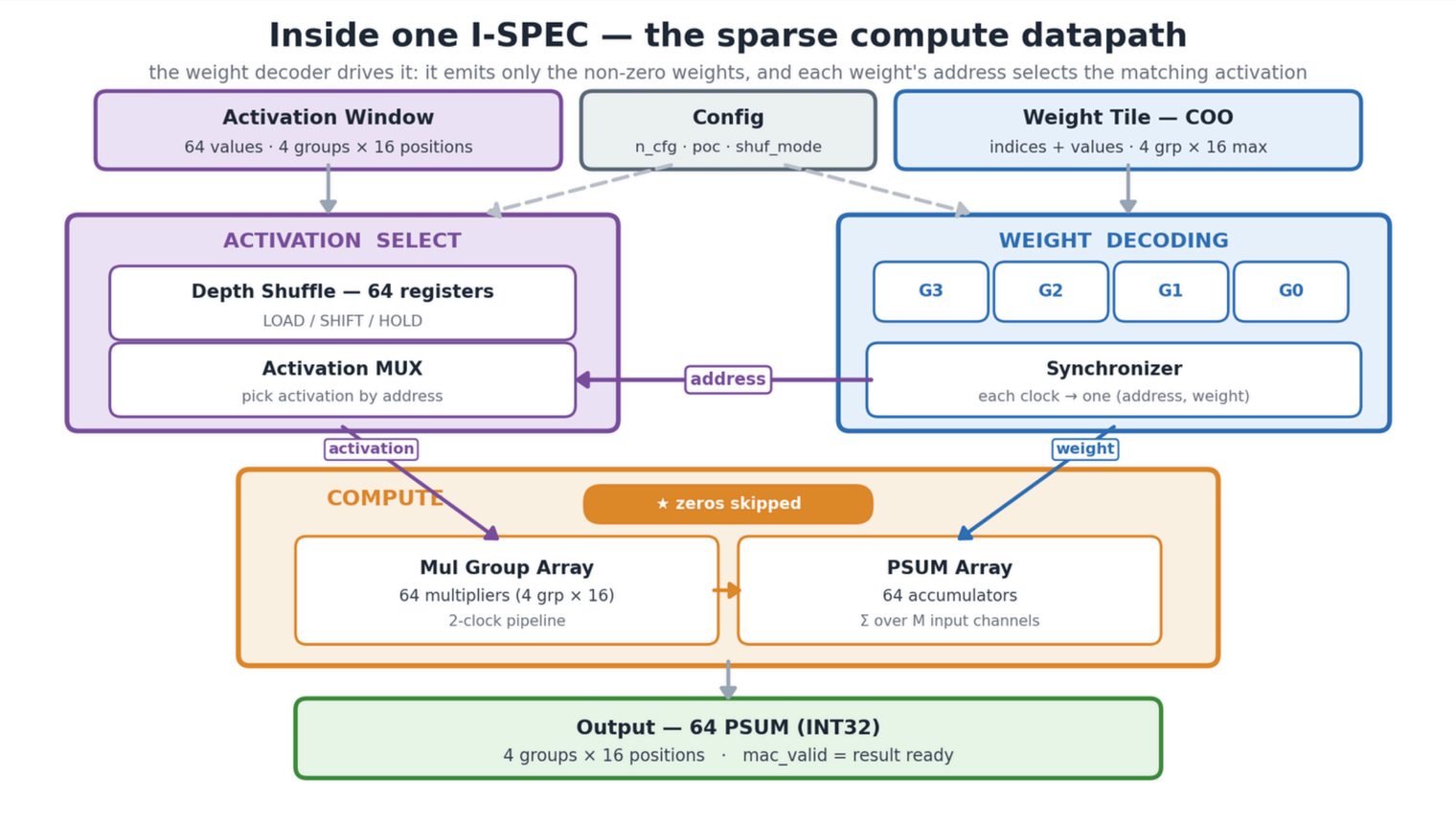




3. Implementation – Accelerator Design

I-SPEC: Sparse Processing Element Cluster

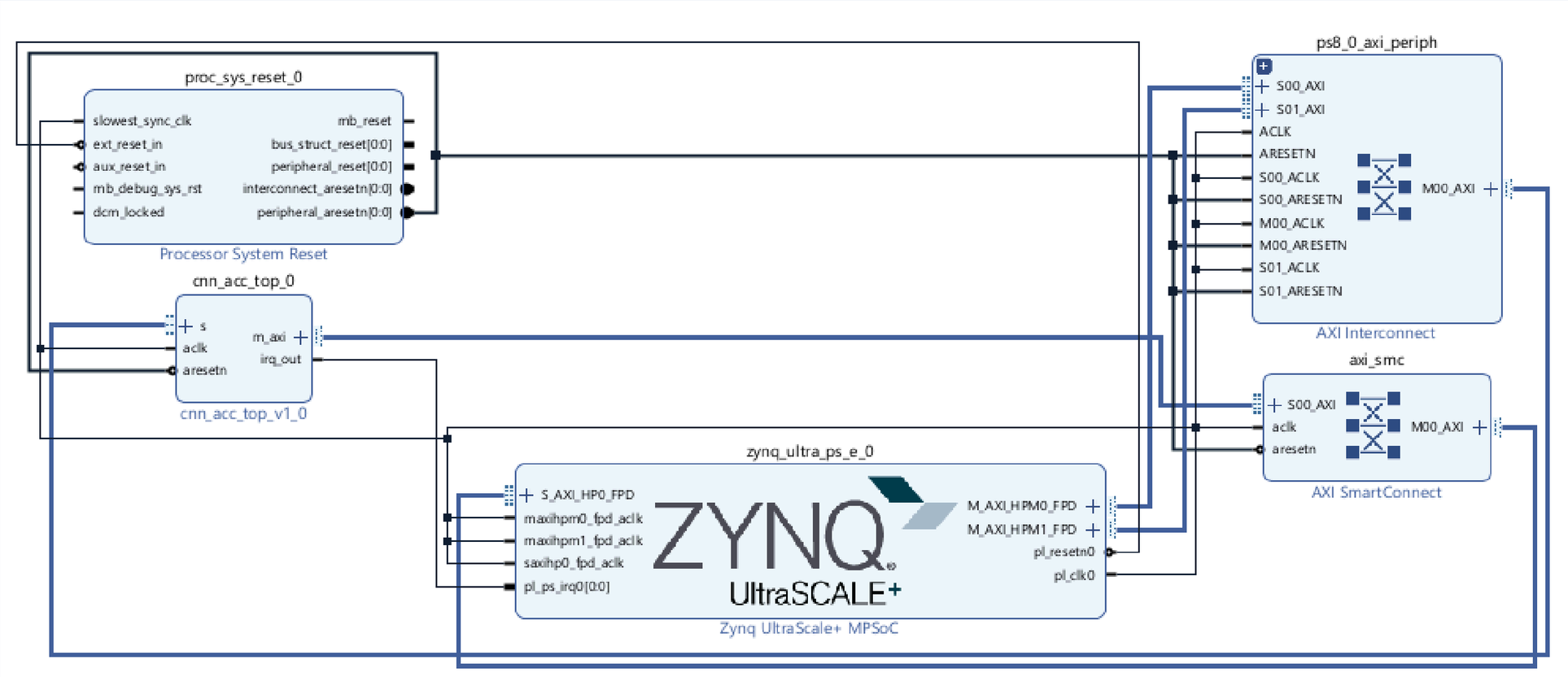
- **I-SPEC** (Input-channel Sparse Processing Element Cluster) is the core compute unit — one weight decoder + PE array per cluster.
- **12 SPEC** clusters × 4 groups × 16 PEs = 768 MAC units total (768 DSP48E2 slices).
- Three levels of parallelism: **SPEC-level** (N non-zero MACs), **Group-level** (G=4 output channels), **Pixel-level** (P=16 spatial positions).
- Sparsity is exploited directly in hardware — idle PEs skip zero-weight multiplications with no wasted cycles.



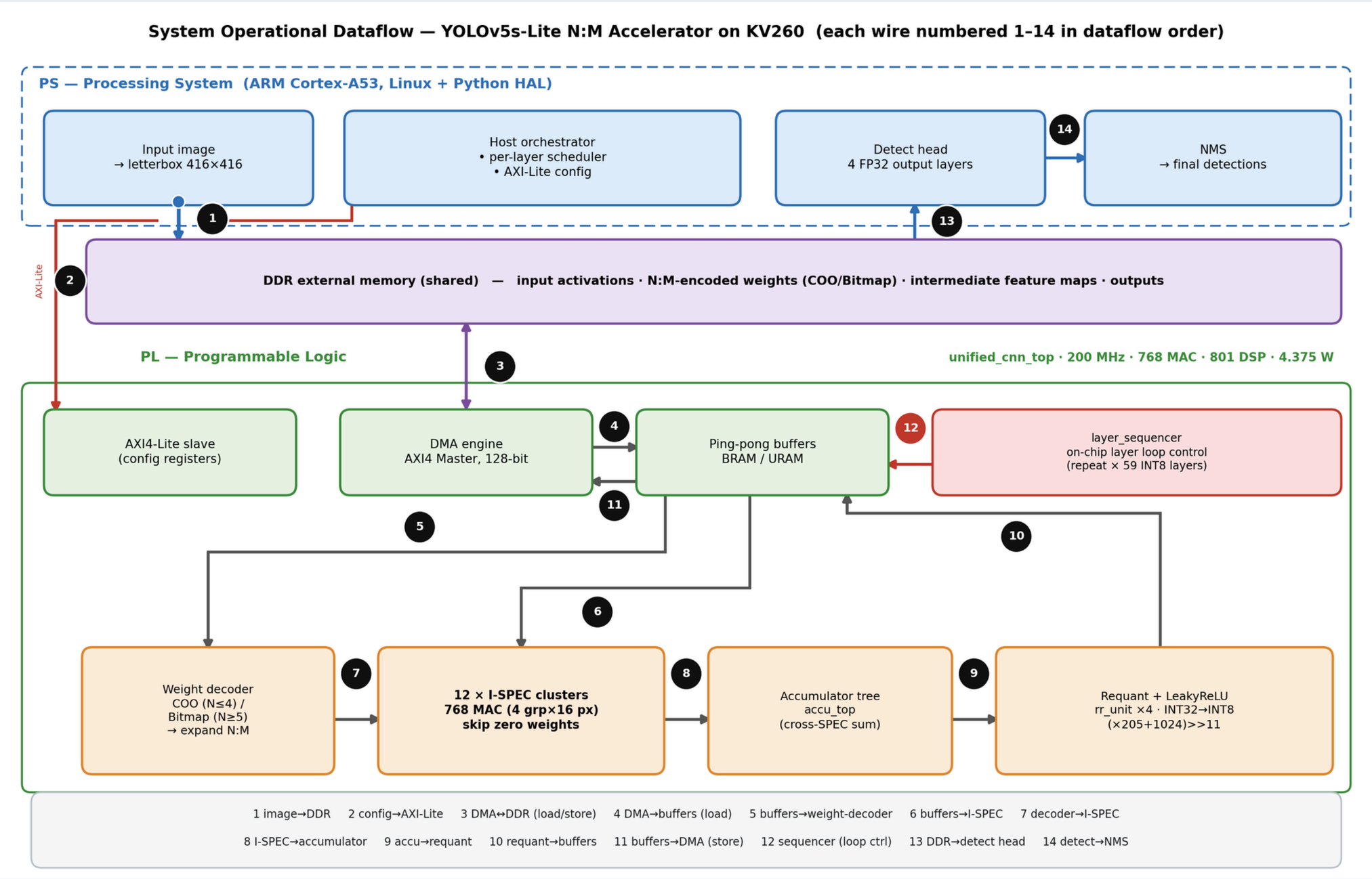
3. Implementation – System Integration

RTL System Integration & KV260 Deployment

- Unified RTL kernel: one datapath handles conv1×1 / conv3×3 / depthwise conv3×3 via a LAYER_TYPE register — 23 modules, ~7,000 lines of hand-written Verilog.
- Packaged as AXI4-Lite (control) + AXI4 Master (128-bit DDR burst) IP, integrated into a Zynq UltraScale+ Vivado block design.
- On-board stack: Linux DMA kernel module + Python HAL drive layer-by-layer inference on the KV260.
- Deployment flow: bitstream + kernel module + weights copied via SCP, then smoke-tested against golden vectors for every layer type before live inference.

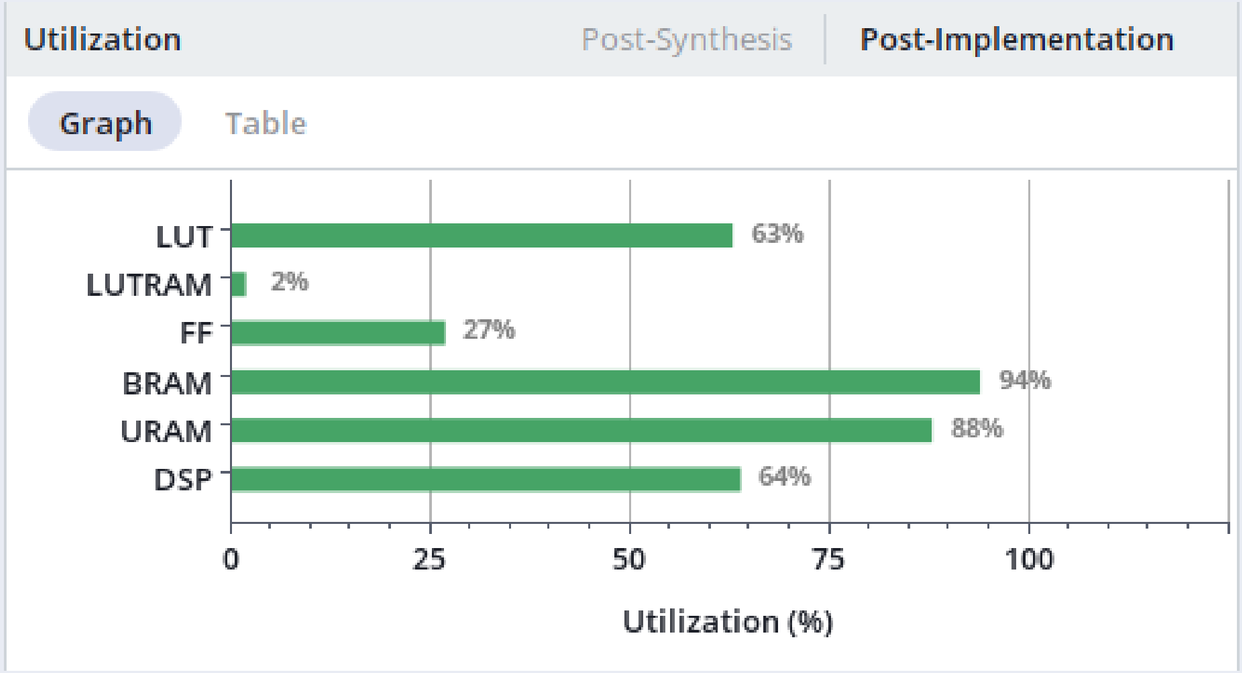


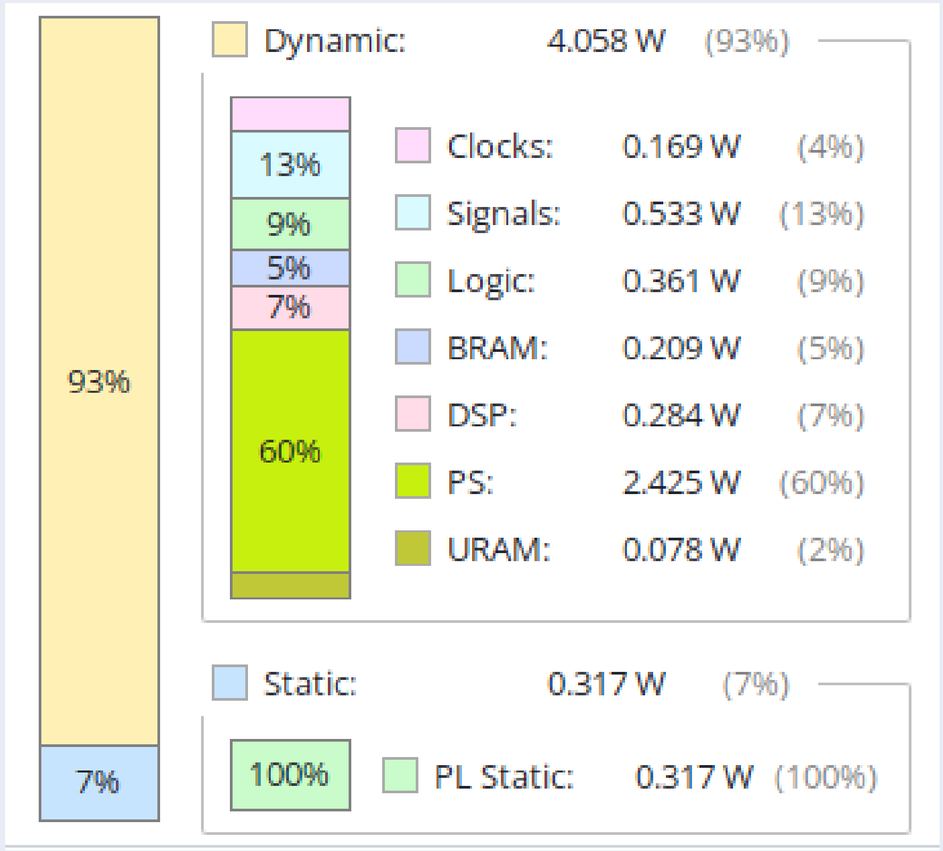
3. Implementation — System Integration



4. Results

FPGA Resource Utilization, Timing & Power





Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	0.004 ns	Worst Hold Slack (WHS)	0.010 ns	Worst Pulse Width Slack (WPWS)	1.000 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	274808	Total Number of Endpoints	274808	Total Number of Endpoints	65481

All user specified timing constraints are met.

4. Results

Bit-Exact Verification & On-Board Detection

- 14 dedicated Verilog testbenches, from mac_unit up to full AXI system simulation.
- 100% element-wise match vs. the Python golden reference across all 59 FPGA-offloaded layers — 0 LSB error. The hardware is bit-exact; the numbers below track what compression itself costs.
- Detection fidelity is now measured at every compression step against the FP32-dense parent on the same test image, instead of a single end-to-end number — see the breakdown at right.
- End-to-end on-board throughput: 1.30 FPS (0.77 s/frame) at 153.6 GOPS peak compute.

100%
layers bit-exact (59/59), 0 LSB error

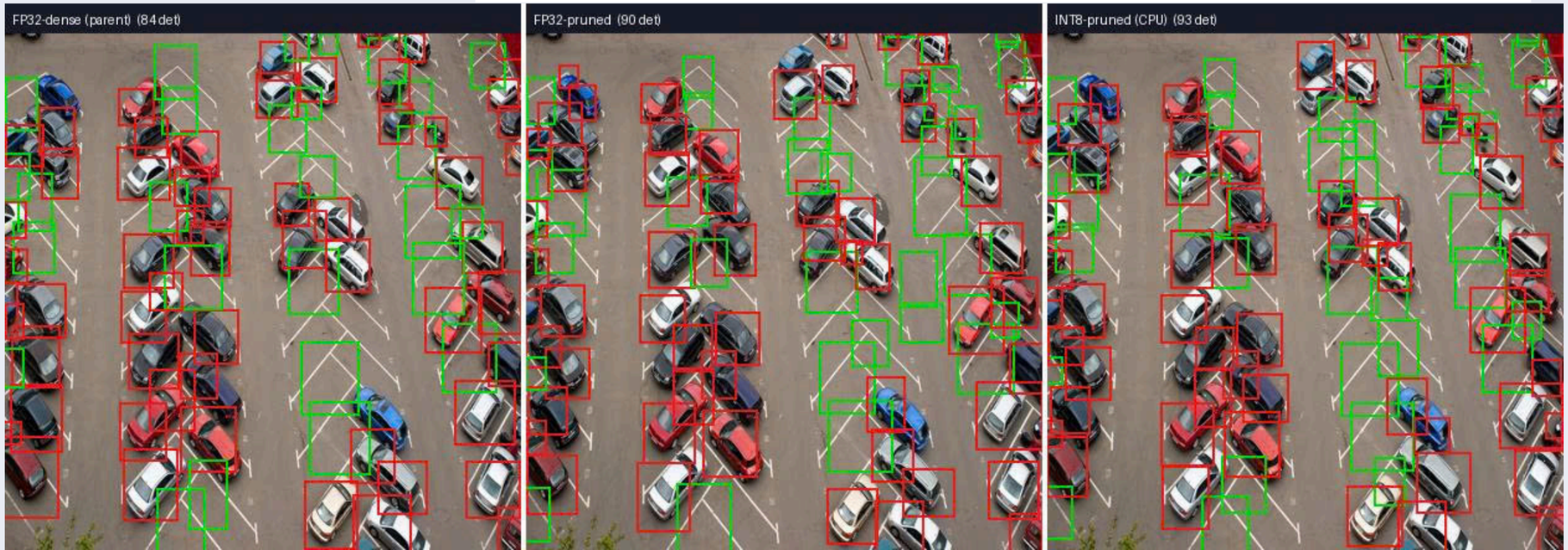
0.600
mean IoU, FP32-dense → INT8-pruned

84→93
detections, dense → INT8-pruned

1.30 FPS
on-board end-to-end throughput

IoU by Compression Step vs. FP32-dense parent, same image

Step	mean	median	≥0.5	≥0.7	≥0.9
PRUNE FP32-dense → FP32-pruned	0.622	0.720	74%	56%	5%
QUANTIZE FP32-pruned → INT8-pruned	**0.756**	**0.871**	88%	69%	32%
TOTAL **FP32-dense → INT8-pruned**	**0.600**	**0.732**	**71%**	**55%**	**4%**



4. Results

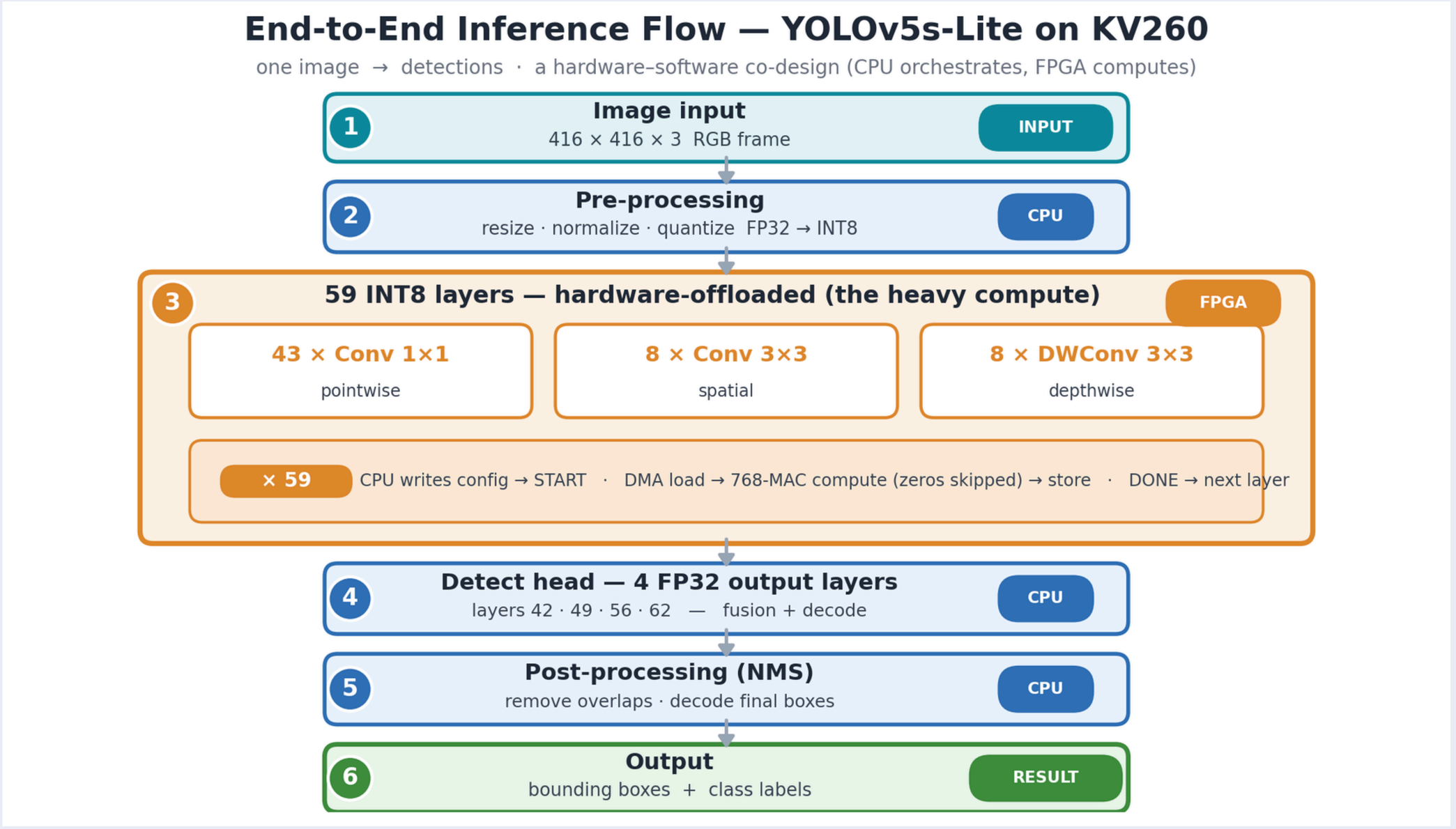
Benchmark Comparison & Limitations

Metric	Xie Et Al. (ResNet-50, ZCU102)	This work (KV260)
MAC units	1,024	768
LUT utilization	236K (84%)	73K (62%)
Power	15.0 W	4.375 W
Peak efficiency	13.7 GOPS/W	~94 GOPS/W

- Direct comparison is inherently limited: model complexity differs (ResNet-50 · 4.1 GFLOP vs. YOLOv5s-Lite · 4.68 GFLOP) and so does platform scale (ZCU102 PL · 274K LUT vs. KV260 PL · 117K LUT).
- DMA/host overhead ≈55% of latency
- Fixed nc=2, 416×416 config
- Detect head runs on ARM CPU
- Validated only on PKLot (2 classes)

5. Conclusions

End-to-end flow



5. Conclusions

Conclusion & Future Work

Objectives

01 ✓ YOLOv5s-Lite designed — 5.15M params (≈1.16M non-zero after N:M)

02 ✓ Compression pipeline ≥10× — achieved ≈40×

03 ✓ Unified RTL accelerator with native N:M decoding

04 ✓ KV260 deployment & 100% bit-exact verification

05 ✓ Timing closure on KV260 — 200 MHz, WNS +0.004 ns

Future Works

01 Double-buffering + C++ runtime → cut DMA idle time by 30–50%.

02 Multi-class support (COCO / VOC) and migration toward YOLOv8.

03 ASIC migration for mass production & higher energy efficiency.